

Viva Q&A — Multi-class Image Segmentation using Deep Learning (segT)

Project: Terrain Safety Analysis with SegFormer

Tech Stack: PyTorch · HuggingFace Transformers · Gradio · Weights & Biases · OpenCV · Plotly

Table of Contents

1. [Project Overview & Objective](#)
 2. [Theoretical Foundations](#)
 3. [SegFormer Architecture \(Deep Dive\)](#)
 4. [Dataset — ADE20k](#)
 5. [System Pipeline & Workflow](#)
 6. [Core Code Walkthrough — `model\_utils.py`](#)
 7. [Depth Estimation — Depth Anything V2](#)
 8. [Safety Mapping & Geometric Refinement](#)
 9. [Pathfinding \(A*\)](#)
 10. [Visualization \(HUD, Depth Overlay, 3D Terrain\)](#)
 11. [Gradio Application — `app.py`](#)
 12. [Training Script — `train\_amp.py`](#)
 13. [Experiment Tracking — Weights & Biases](#)
 14. [Deployment](#)
 15. [Limitations, Future Work & Ethics](#)
 16. [Rapid Fire / Short Answers](#)
-

1. Project Overview & Objective

Q1. What is the main objective of your project?

A. The project builds a **real-time terrain safety analysis system** that takes an image of any outdoor/indoor scene, segments every pixel into one of 150 semantic classes using a **SegFormer** model, and then maps those classes into **Safe** (green), **Hazard** (red), or **Neutral** (transparent) categories. It outputs a **Safety Score (%)**, a color-coded **HUD overlay**, an optional **safe navigation path**, and an interactive **3D terrain mesh**. The aim is to assist autonomous navigation, search-and-rescue drones, or visually impaired users.

Q2. What problem does this project solve?

A. Navigating complex environments requires understanding **what** is on the ground at every pixel, not just detecting objects. Traditional object detectors give bounding boxes; they cannot tell you "the left 40% of the ground is grass (safe) and the right 60% is water (hazard)." Semantic segmentation solves this by giving a **per-pixel class label**, and our system further abstracts it into an actionable binary safety map.

Q3. What is the title of your project report?

A. "Multi-class Image Segmentation using Deep Learning."

Q4. What are the key features of the system?

A.

1. **SegFormer B0/B2** inference for 150-class segmentation.
2. **Monocular Depth Estimation** via Depth Anything V2.
3. **Safety Score** computation from safe/hazard pixel ratios.
4. **Geometric Refinement** — slope-based override to fix false hazards (e.g., flat gravel classified as rock).
5. **A* Pathfinding** — finds safe path from bottom-center to topmost safe pixel.

6. **HUD Overlay** — green/red transparency overlay on original image.
7. **3D Sci-Fi Terrain Mesh** — interactive Plotly visualization.
8. **W&B Logging** — every inference is logged with images, metrics, and a structured table.
9. **Gradio web UI** — interactive, deployable on Hugging Face Spaces.
10. **Configurable class mapping** — users can re-define what is "safe" vs "hazard" via JSON at runtime.

Q5. Who are the intended users?

A. Developers, hobbyists, and student researchers. It is **not** intended for safety-critical systems like autonomous driving on public roads.

2. Theoretical Foundations

Q6. What is Semantic Segmentation?

A. Semantic segmentation is the task of assigning a **class label to every pixel** in an image. Unlike classification (one label per image) or detection (bounding boxes), segmentation produces a dense pixel-wise map. For example, in a street scene, every pixel of the road is labeled "road", every pixel of a tree is labeled "tree", etc.

Q7. How is semantic segmentation different from instance segmentation and panoptic segmentation?

A.

Type	What it does	Example
Semantic	Labels every pixel with a class; doesn't distinguish <i>instances</i>	All cars → same color
Instance	Detects individual objects with masks; background ignored	Car-1, Car-2 each get unique mask
Panoptic	Combines both — every pixel gets a class <i>and</i> instance ID	Road=stuff, Car-1=thing-1, Car-2=thing-2

Our project uses **semantic segmentation** since we care about region types (grass vs water), not individual object instances.

Q8. What was the evolution of segmentation models before SegFormer?

A.

1. **FCN (2015)** — First to use fully convolutional layers for dense prediction, replacing the final FC layers of classification CNNs.
2. **U-Net (2015)** — Encoder-decoder with skip connections, originally for biomedical.
3. **DeepLabV3+ (2018)** — Atrous/dilated convolutions + Atrous Spatial Pyramid Pooling (ASPP) for multi-scale context.
4. **Vision Transformers (ViT, 2020)** — Applied self-attention to image patches, capturing global context.
5. **SegFormer (2021)** — A **hierarchical Transformer** encoder with a lightweight MLP decoder, designed specifically for efficient segmentation.

Q9. What is the key limitation of CNNs that Transformers overcome?

A. CNNs use **local convolution kernels** (e.g., 3×3), so their receptive field grows slowly with depth. This limits their ability to model **long-range dependencies** (e.g., understanding that a distant patch of blue is part of the same lake). Transformers use **self-attention**, which relates every patch to every other patch in a single layer, giving **global context** at every level.

Q10. What is self-attention and how does it work?

A. Self-attention computes a weighted sum over all positions in a sequence. Given input tokens, it produces three vectors per token: **Query (Q)**, **Key (K)**, and **Value (V)**. The attention score between token i and j is:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

Each token "attends" to all other tokens with learned relevance weights. In vision, each image patch attends to every other patch, enabling global reasoning.

3. SegFormer Architecture (Deep Dive)

Q11. What is SegFormer? Who proposed it?

A. SegFormer is a **Transformer-based semantic segmentation framework** proposed by **Xie et al. (2021)** from NVIDIA. It consists of a **hierarchical Transformer encoder** (Mix Transformer / MiT) and a lightweight **All-MLP decoder**. It achieves strong accuracy with high efficiency.

Q12. Describe the SegFormer encoder (Mix Transformer — MiT).

A. The encoder has **4 stages**, each producing feature maps at decreasing resolution:

- **Stage 1:** 1/4 resolution, captures fine details.
- **Stage 2:** 1/8 resolution.
- **Stage 3:** 1/16 resolution.
- **Stage 4:** 1/32 resolution, captures high-level semantic info.

Each stage contains:

1. **Overlap Patch Embedding** — Uses overlapping convolutions (not standard non-overlapping ViT patches) to produce tokens. This preserves local continuity.
2. **Efficient Self-Attention** — Reduces the spatial resolution of K and V by a factor R (spatial reduction ratio), making attention $O(N^2/R^2)$ instead of $O(N^2)$, dramatically cutting compute.
3. **Mix-FFN** — A feed-forward network that includes a **3×3 depthwise convolution** inside it. This injects local positional information implicitly, **eliminating the need for explicit positional encodings** — a key differentiator from ViT.

Q13. Why does SegFormer not use positional encodings?

A. Standard ViT uses fixed or learnable positional encodings (sinusoidal or learned vectors added to patch embeddings). These are resolution-dependent: if you train on 512×512 and test on 1024×1024, positional encodings break. SegFormer instead uses **Mix-FFN** with a depthwise convolution that implicitly encodes spatial position via local convolution, making it **resolution-agnostic**.

Q14. Describe the SegFormer decoder.

A. The decoder is an **All-MLP** (multi-layer perceptron) design:

1. Takes multi-scale features from all 4 encoder stages (F1, F2, F3, F4).
2. **Upsamples** each to 1/4 resolution using bilinear interpolation.
3. **Projects** each to a common channel dimension via a linear layer.
4. **Concatenates** all four.
5. Passes through a **fusion MLP** to produce the final per-pixel class predictions.

This is much lighter than the complex decoders in DeepLab (ASPP) or U-Net (transposed convolutions), contributing to SegFormer's speed.

Q15. What are the B0 and B2 variants? What's the difference?

A. SegFormer comes in variants **B0 to B5**, scaling the encoder width and depth:

Variant	Params (approx)	mIoU on ADE20k	Speed
B0	~3.7M	~37-38%	Fastest (CPU-friendly)
B2	~27M	~46-47%	Balanced
B5	~85M	~51%+	Slowest

Our project offers **B0** (fast, default) and **B2** (higher accuracy) as user-selectable options.

Q16. What is mIoU? How is it calculated?

A. **Mean Intersection over Union (mIoU)** is the standard metric for semantic segmentation. For each class c :

$$\text{IoU}_c = \text{TP}_c / (\text{TP}_c + \text{FP}_c + \text{FN}_c)$$

Where TP=true positives, FP=false positives, FN=false negatives (all at pixel level). Then:

$$\text{mIoU} = (1/C) \times \sum \text{IoU}_c$$

It averages IoU across all C classes. A B0 model achieving 37% mIoU means that on average across 150 ADE20k classes, the predicted and ground-truth regions for each class overlap by 37%.

Q17. What input resolution does the model expect?

A. The SegFormer models used are fine-tuned at **512×512** pixels. The `SegformerFeatureExtractor` resizes and normalizes the input internally. However, the **logits are then bilinearly upsampled** back to the **original image size** in our code, so the output mask matches the input dimensions exactly.

4. Dataset — ADE20k

Q18. What dataset is the model trained on?

A. The pre-trained SegFormer models are fine-tuned on **ADE20k**, a scene parsing benchmark from MIT CSAIL.

Q19. Describe the ADE20k dataset.

- A.
- **Authors:** Bolei Zhou, Hang Zhao, et al. (CVPR 2017).
 - **Scale:** ~20,000 training images, ~2,000 validation images.
 - **Classes:** 150 semantic categories covering diverse objects and stuff (wall, sky, floor, tree, car, person, grass, water, building, etc.).
 - **Diversity:** Covers indoor rooms, outdoor landscapes, street scenes, and natural environments.
 - **Annotation:** Dense pixel-level labeling for all 150 classes.

Q20. Did you collect or label any custom data?

A. No. We use the **pre-trained weights** from NVIDIA's HuggingFace model hub (`nvidia/segformer-b0-finetuned-ade-512-512`). For inference, users upload their own images dynamically. These are logged to W&B as a running "test set" of real-world images.

Q21. How do you map 150 classes to just Safe/Hazard?

A. We define two keyword lists:

- **Safe:** grass, road, dirt, floor, path, vegetation, earth, field, plant
- **Hazard:** rock, water, sea, river, lake, pool, waterfall, boulder, cliff, person, vehicle, car, truck, bus, train, motorcycle, bicycle, snow, ice, mountain, hill

For each of the 150 class labels, we check if the label string **contains** any safe or hazard keyword. If it matches a hazard keyword, it's hazard (hazard takes priority). If it matches only a safe keyword, it's safe. Everything else is **neutral** (ignored/transparent). This mapping is **user-editable** at runtime via a JSON editor in the UI.

5. System Pipeline & Workflow

Q22. Describe the end-to-end pipeline of your system.

A.

Image Upload → Feature Extraction → SegFormer Inference → Logit Upsampling → Argmax →
 Raw Mask (150 classes) → Safety Mapping (Safe/Hazard/Neutral) →
 [Optional] Depth Estimation → Geometric Refinement (Slope override) →
 [Optional] A* Pathfinding →
 HUD Overlay + Depth Overlay + 3D Terrain Mesh + Safety Score + Stats →
 W&B Logging → Display in Gradio UI

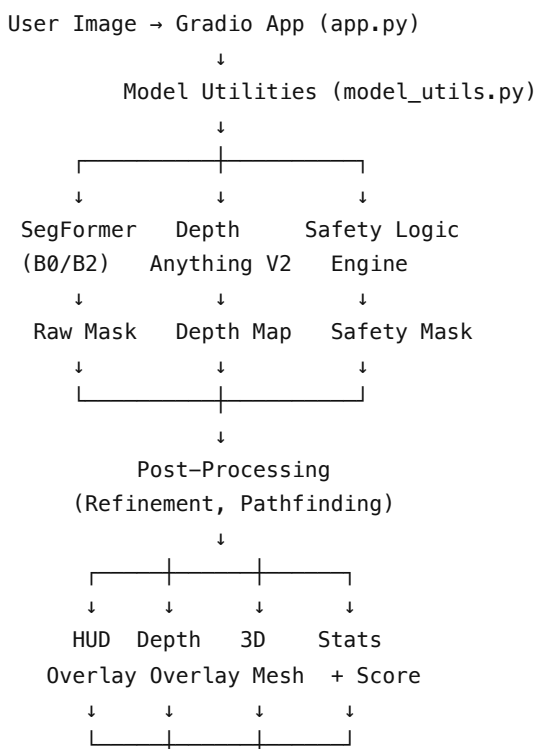
Q23. Explain each step in detail.

A.

1. **Image Upload:** User uploads a PIL Image via Gradio.
2. **Feature Extraction:** `SegformerFeatureExtractor` resizes to 512×512, normalizes pixel values, and converts to a PyTorch tensor.
3. **Inference:** Forward pass through the SegFormer model produces raw **logits** of shape `(1, 150, H/4, W/4)` — one score per class per spatial location at 1/4 resolution.
4. **Upsampling:** Logits are bilinearly upsampled to the **original image size** using `torch.nn.functional.interpolate`.
5. **Argmax:** `argmax(dim=1)` across the 150 class channels gives the predicted class ID for each pixel → produces a 2D integer mask.
6. **Safety Mapping:** Each class ID is looked up against the safe/hazard keyword lists to produce a `safety_mask` where 0=Neutral, 1=Safe, 2=Hazard.
7. **Depth Estimation** (optional): Depth Anything V2 produces a normalized depth map (0 to 1).
8. **Geometric Refinement** (optional): Morphological closing fills small gaps in safe regions. Slope-based override re-classifies flat "hazard" pixels (like gravel) as safe.
9. **Pathfinding** (optional): A* search from bottom-center to topmost safe pixel using a cost map derived from the safety mask and slope.
10. **Visualization:** HUD overlay (green/red), depth heatmap overlay (turbo colormap), and 3D Plotly mesh are generated.
11. **Stats:** Safety Score, pixel counts, top-1 class, mean confidence are computed.
12. **W&B Log:** All metrics + side-by-side images are logged.

Q24. What is the architecture diagram of the system?

A.



6. Core Code Walkthrough — `model_utils.py`

Q25. What does `load_model()` do?

A. It loads a SegFormer model and its feature extractor from HuggingFace, moves it to the available device (CUDA GPU or CPU), sets it to `eval()` mode, and **caches** it in a global `MODEL_CACHE` dictionary. Subsequent calls with the same model name return the cached object, avoiding re-downloading and re-loading. This is critical for a web app where many requests may come in.

Q26. How is the segmentation mask predicted? Explain `predict_mask()`.

A.

```
inputs = feature_extractor(images=image, return_tensors="pt") # Preprocess
inputs = {k: v.to(device) for k, v in inputs.items()}         # Move to GPU
with torch.no_grad():                                         # No gradients
    outputs = model(*inputs)                                   # Forward pass
logits = outputs.logits                                       # (1, 150, H/4, W/4)
upsampled = F.interpolate(logits, size=image.size[:-1], mode="bilinear") # Upsample
pred_seg = upsampled.argmax(dim=1)[0]                         # (H, W) integer mask
```

Key decisions:

- `torch.no_grad()` disables gradient computation → saves memory and speeds up inference.
- Bilinear interpolation of **logits** (not the mask) before `argmax` is the **correct standard practice** — it preserves class boundary sharpness.
- PIL gives `(W, H)`, PyTorch expects `(H, W)`, hence `image.size[:-1]`.

Q27. Explain the `map_classes_to_safety()` function in detail.

A. This function takes the raw integer mask and the model's `id2label` mapping (e.g., `{0: "wall", 9: "grass", ...}`). For each class ID:

1. Clean the label string (lowercase, strip whitespace).
2. Check if any safe keyword is a **substring** of the label (e.g., "grass" matches "grassland").
3. Check if any hazard keyword is a substring.
4. If it matches hazard → mark as hazard (priority). If only safe → mark as safe. Otherwise → neutral (0).
5. Uses `np.isin()` for vectorized mask creation — efficient for large images.

Output: `safety_mask` array where 0=Neutral, 1=Safe, 2=Hazard.

Q28. What is the model caching strategy and why is it important?

A. A global dictionary `MODEL_CACHE = {}` stores loaded models keyed by model name/ID. On subsequent calls, the model is served from memory. This is essential because:

- Loading a model from HuggingFace involves downloading weights (~15MB for B0) and initializing the PyTorch model, which can take several seconds.
- In a web app (Gradio), multiple user requests would otherwise trigger redundant loads.
- The cache persists for the lifetime of the Python process.

Q29. Why is `torch.no_grad()` used during inference?

A. During inference, we don't need to compute gradients (no backpropagation). `torch.no_grad()`:

1. **Saves memory** — PyTorch doesn't store intermediate activations needed for backward pass.
2. **Speeds up computation** — Skips gradient graph construction.

3. **Best practice** for any production inference code.

Q30. Why do you upsample logits with bilinear interpolation and then argmax, instead of the reverse?

A. If you argmax first (producing an integer mask) and then resize with bilinear interpolation, the interpolation would **blend class IDs** (e.g., averaging class 5 and class 12 → class 8, which is meaningless). Nearest-neighbor could be used for integer masks, but it produces blocky boundaries. The standard approach is:

1. Bilinear upsample the **floating-point logits** (preserves smooth gradients between classes).
2. Then argmax → produces sharp, high-resolution class boundaries.

7. Depth Estimation — Depth Anything V2

Q31. What is monocular depth estimation?

A. It is the task of predicting **relative depth values** for every pixel in a single RGB image — estimating which parts of the scene are closer or farther. Unlike stereo depth (two cameras) or LiDAR, monocular depth uses **a single camera** and relies on learned cues (perspective, texture gradients, occlusion).

Q32. What model do you use for depth estimation?

A. **Depth Anything V2** from the HuggingFace model hub. Two sizes are available:

- `depth-anything/Depth-Anything-V2-Small-hf` (fast)
- `depth-anything/Depth-Anything-V2-Base-hf` (higher quality)

It is loaded via HuggingFace's `pipeline(task="depth-estimation")`.

Q33. How is the depth map normalized?

A. The raw depth output is a PIL Image. It is converted to a numpy array, then **min-max normalized** to the range [0, 1]:

```
depth_norm = (depth_np - depth_min) / (depth_max - depth_min)
```

This produces a relative depth map where 0 = farthest and 1 = nearest (or vice versa depending on the model's convention).

Q34. How is depth used in your project?

A. Depth serves three purposes:

1. **Geometric Refinement**: Compute slope (gradient of depth). If a "hazard" pixel (e.g., rock) is on flat ground (slope < threshold), override to "safe" — because flat rocks/gravel are walkable.
2. **Pathfinding Cost**: High-slope regions get a heavy penalty in the cost map, so the A* path avoids steep terrain.
3. **3D Terrain Visualization**: Depth values become Z-coordinates for the 3D mesh surface.

8. Safety Mapping & Geometric Refinement

Q35. What is the Safety Score and how is it calculated?

A.

```
Safety Score = 100 × (safe_pixels / total_pixels)
```

For example, if 60% of pixels are classified as safe, the score is 60%. It ignores neutral pixels in the numerator but includes them in the denominator, so a scene with mostly sky (neutral) and some grass (safe) will have a relatively low score.

Q36. What is geometric refinement? Why is it needed?

A. The SegFormer model is purely **semantic** — it classifies pixels based on visual appearance. This causes problems:

- Flat gravel → classified as "rock" (hazard) → but it's perfectly walkable.
- Mountain ridges → correctly "mountain" (hazard) → not walkable.

Both are "rock/mountain" semantically, but they have very different **geometry** (flat vs steep). Geometric refinement uses depth to compute **slope** and overrides flat "hazard" pixels to "safe."

Q37. Explain `refine_safety_mask()` in detail.

A. Two strategies:

1. **Morphological Closing** (`cv2.MORPH_CLOSE` with 5×5 kernel): Fills small gaps in safe regions. If scattered safe pixels exist near each other, closing connects them into a contiguous region.
2. **Slope-Based Override**:
 - Compute depth gradient: `gy, gx = np.gradient(gaussian_filter(depth, sigma=2))`
 - Compute slope magnitude: `slope = sqrt(gx2 + gy2)`
 - If a pixel is marked Hazard (2) AND its slope < 0.05 (flat), override to Safe (1).
 - Sigma=2 Gaussian smoothing is applied first to ignore pebble-level noise.

Q38. Why is the slope threshold set to 0.05?

A. This is an **empirical parameter** tuned through experimentation. A normalized depth map ranges 0–1, so gradient values are typically small. 0.05 was found to be a good balance: it catches bumpy gravel as "flat" (walkable) while still keeping steep mountain faces as "hazard." It was increased from an earlier value after finding that gravel was still being flagged.

9. Pathfinding (A*)

Q39. How does the pathfinding algorithm work?

A. The function `compute_path()` uses `skimage.graph.route_through_array`, which implements a **minimum-cost path** algorithm (equivalent to Dijkstra/A* on a grid). Steps:

1. **Cost Map**: Safe pixels → cost 1. Non-safe pixels → cost 200. If depth available, add slope penalty: high-slope regions get +1000 cost, plus a continuous `slope × 100` penalty.
2. **Start**: Bottom-center of the image `(H-1, W//2)`.
3. **End**: The **topmost safe pixel** (minimum row index among all safe pixels).
4. The algorithm finds the minimum-cost path through the 2D cost grid.
5. **Validation**: If the average cost per step exceeds 800, the path likely traversed too much hazard → return `None` ("No Safe Path Found").

Q40. Why is the hazard cost set to 200 and not infinity?

A. Using infinity would make it impossible to traverse hazard at all, and the algorithm would fail if the goal is surrounded by any hazard. A **soft cost** of 200 (previously 50, then increased) means the algorithm strongly prefers safe terrain but can cross small hazard patches if there's no alternative. This produces more realistic paths — "avoid the lake if possible, but cross a narrow stream if you must."

Q41. Why was the cost increased from 50 to 200?

A. With cost=50, the path would sometimes **climb over mountains** because the total cost of the short hazard crossing was lower than the long safe detour. At cost=200, the "long way around" becomes preferable, forcing the path to find genuinely safe routes.

Q42. How does slope affect pathfinding?

A. In addition to the safety-based cost, the slope (from depth gradient) adds:

- `+1000` for pixels with slope > 0.05 (steep).
- `+slope × 100` continuously for all pixels.

This ensures the path avoids steep terrain even if it's semantically labeled "safe" (e.g., a steep grassy hillside).

Q43. What is the start and end point strategy?

A.

- **Start:** Always bottom-center of the image `(H-1, W//2)` — simulating a person standing at the bottom of the frame.
 - **End:** Dynamically chosen as the **topmost (minimum row) safe pixel** in the image — representing the farthest reachable safe point. If no safe pixels exist, defaults to top-center `(0, W//2)`.
-

10. Visualization (HUD, Depth Overlay, 3D Terrain)

Q44. How does the HUD overlay work?

A. The `create_hud()` function:

1. Creates a blank overlay array same size as the image.
2. Sets safe pixels to green `(0, 255, 0)` and hazard pixels to red `(255, 0, 0)`.
3. Alpha-blends the overlay onto the original image: `blended = image * (1 -  $\alpha$ ) + overlay *  $\alpha$` , where `$\alpha = 0.4$` by default.
4. Draws the path as a thick blue line with a lighter "glow" underneath and white dot arrows every 15 points.

Q45. How is the depth overlay created?

A. The `create_depth_overlay()` function:

1. Normalizes the depth map to 0–255 uint8.
2. Applies OpenCV's **TURBO colormap** (Blue=Far, Red=Near) to create a heatmap.
3. Converts BGR→RGB (OpenCV convention).
4. Blends with the original image using `cv2.addWeighted()`.
5. Optionally draws the path in **cyan** for high contrast on the heatmap.

Q46. Describe the 3D terrain visualization.

A. The `create_3d_terrain()` function creates an interactive Plotly 3D mesh:

1. **Downsamples** the depth map by stride=4 for performance.
2. Uses depth values as **Z-coordinates** (height).
3. Colors the surface based on the safety mask: Gray=Neutral, Green=Safe, Red=Hazard.
4. Adds **white wireframe contour lines** for a sci-fi aesthetic.
5. If a path exists, renders it as a **cyan 3D line** floating slightly above the surface.
6. Uses dark background, hidden axes, manual aspect ratio (`z=0.3*x`) to prevent the terrain from looking flat.

Q47. Why is stride=4 used in 3D visualization?

A. A Plotly Surface mesh at full resolution (e.g., $512 \times 512 = 262,144$ vertices) would be extremely slow to render in the browser. Stride=4 reduces it to $\sim 128 \times 128 = \sim 16,384$ vertices, which is fast enough for interactive rotation while still showing the terrain shape.

11. Gradio Application — `app.py`

Q48. What is Gradio and why did you choose it?

A. Gradio is a Python library to build **interactive web UIs** for ML models with minimal code. We chose it because:

1. **Rapid prototyping** — a complete UI in ~70 lines.
2. **Built-in image/JSON/plot components** — no HTML/JS needed.
3. **Hugging Face Spaces integration** — one-click cloud deployment.
4. **Queue support** — handles concurrent users.

Q49. What are the user-configurable parameters in the UI?

A.

Parameter	Type	Default	Purpose
Model Version	Dropdown	B0 (Fast)	Choose SegFormer B0 or B2
HUD Opacity	Slider 0–1	0.4	Overlay transparency
Class Mapping JSON	Code editor	Default safe/hazard lists	Customize safety definitions
Enable Depth	Checkbox	Off	Toggle depth estimation
Depth Model	Dropdown	Small (Fast)	Depth Anything V2 size
Depth Overlay Opacity	Slider 0–1	0.5	Depth heatmap transparency
Enable Pathfinding	Checkbox	Off	Toggle A* path
Show 3D View	Checkbox	Off	Toggle 3D terrain mesh

Q50. How does `process_image()` in `app.py` orchestrate the pipeline?

A. It is the main callback wired to the "Analyze Terrain" button:

1. Loads the selected model (cached).
2. Parses the JSON mapping config.
3. Runs segmentation inference (timed).
4. Computes safety mask from the raw mask.
5. Optionally runs depth estimation.
6. Optionally refines the safety mask with geometry.
7. Optionally runs pathfinding.
8. Generates HUD overlay, depth overlay, and 3D plot.
9. Computes stats (safety score, pixel counts, confidence).
10. Logs everything to W&B.
11. Returns 6 outputs to the UI: HUD image, depth image, raw mask, JSON stats, score label, 3D plot.

Q51. What outputs does the Gradio app display?

A. Six outputs:

1. **HUD Prediction** — green/red overlay with path drawn.
2. **Depth Overlay** — heatmap of estimated depth.
3. **Raw Mask** — the integer segmentation mask (colored palette).
4. **Detailed Stats (JSON)** — safety score, pixel counts, top-1 class, mean confidence.
5. **Safety Score (Label)** — e.g., "Safety Score: 73.21%"
6. **3D Terrain View (Plot)** — interactive Plotly mesh.

12. Training Script — `train_amp.py`

Q52. What is `train_amp.py` ?

A. It's an **isolated, optional** training script that demonstrates how one would fine-tune the SegFormer model on a custom dataset. It is completely independent of `app.py` and `model_utils.py` (no imports between them) to avoid circular dependencies.

Q53. What is AMP and why is it used?

A. **Automatic Mixed Precision (AMP)** uses both 16-bit (FP16) and 32-bit (FP32) floating point during training. Benefits:

- **~2× speedup** on GPUs with Tensor Cores (NVIDIA Volta+).

- **~50% less GPU memory** for activations.
- Minimal accuracy loss (the loss and critical operations stay in FP32).

In the code:

```
scaler = torch.cuda.amp.GradScaler()           # Scales loss to prevent underflow in FP16
with torch.cuda.amp.autocast():                # Context manager for FP16 forward pass
    outputs = model(pixel_values, labels)
scaler.scale(loss).backward()                  # Scaled backward pass
scaler.step(optimizer)                         # Unscales and steps
scaler.update()                               # Adjusts scale factor
```

Q54. What is the DummyDataset?

A. A placeholder `torch.utils.data.Dataset` that generates **random tensors** (3×512×512 images and 512×512 label maps with random class IDs 0–149). It exists purely for demonstration — in real fine-tuning, you'd load actual labeled images from disk, e.g., a custom terrain dataset.

Q55. How are checkpoints saved and tracked?

A. After each epoch:

1. `torch.save(model.state_dict(), "checkpoints/model_epoch_{N}.pth")` — saves weights to disk.
2. A **W&B Artifact** is created and logged: `wandb.Artifact(f"model-epoch-{N}", type="model")`. This registers the checkpoint in W&B's version control system, enabling lineage tracking and rollback.

Q56. What optimizer and learning rate are used?

A. **AdamW** optimizer with a default learning rate of **5e-5** (0.00005). AdamW is the standard for fine-tuning Transformer models — it decouples weight decay from the gradient update, leading to better generalization.

13. Experiment Tracking — Weights & Biases

Q57. What is Weights & Biases (W&B) and why do you use it?

A. W&B is a cloud-based experiment tracking platform. In this project, it is used for:

1. **Logging metrics:** Safety score, inference time, safe/hazard percentages per image.
2. **Logging media:** Side-by-side original + HUD images with score captions.
3. **Structured tables:** A `wandb.Table` accumulates rows (one per inference) with model ID, scores, confidence, and image references.
4. **System config:** OS, Python version, PyTorch version, device (CPU/GPU) are recorded.
5. **Training artifacts:** Checkpoints are versioned as W&B Artifacts.

Q58. What is logged on each inference?

A.

- **Scalars:** `safety_score`, `safe_pct`, `hazard_pct`, `time_ms`, `mean_conf`, `top1_class`.
- **Image:** A combined image (original | HUD) with a caption showing score, hazard %, and time.
- **Table Row:** model, score, safe%, hazard%, time, top class, confidence, HUD image reference.

Q59. How is the W&B run initialized?

A. In `app.py` :

```
wandb.init(project="terrain-safety-v1", job_type="inference", config=APP_CONFIG)
```

`APP_CONFIG` includes system OS, Python version, Torch version, and device. If W&B init fails (e.g., no API key), it prints a warning and logging is silently disabled — the app still functions.

14. Deployment

Q60. How is the app deployed?

A. The app is designed for deployment on **Hugging Face Spaces** (Gradio SDK). Steps:

1. Create a new Space on huggingface.co with SDK = Gradio.
2. Upload: `app.py` , `model_utils.py` , `requirements.txt` .
3. Optionally set `WANDB_API_KEY` as a Space secret.
4. The Space auto-builds and launches. On first cold boot, it downloads the SegFormer model weights (~15MB).

Locally, it runs via `python app.py` → `http://127.0.0.1:7860` .

Q61. What hardware does it need?

A.

- **Minimum:** Multi-core CPU, 8GB RAM. B0 runs at ~100–300ms per image on CPU.
 - **Recommended:** NVIDIA GPU with CUDA and 4GB+ VRAM for <50ms inference with B2.
 - **Hugging Face Free Tier:** CPU Basic tier works with B0.
-

15. Limitations, Future Work & Ethics

Q62. What are the limitations of this project?

A.

1. **Training data bias:** ADE20k covers general scenes. Exotic terrains (deep snow, underwater, alien landscapes) are poorly represented.
2. **Lighting sensitivity:** Performance degrades significantly at night or under extreme lighting.
3. **Semantic ambiguity:** Flat rocks misclassified as hazards (partially addressed by geometric refinement, but not fully solved).
4. **No temporal reasoning:** Each frame is analyzed independently — no tracking or temporal smoothing for video.
5. **Depth is relative:** Depth Anything V2 produces relative depth (not absolute metric depth), so actual distances cannot be computed.
6. **Safety mapping is heuristic:** The safe/hazard keyword matching is a substring check, which may cause false matches (e.g., "waterfall" matches "water").

Q63. What future work do you propose?

A.

1. **Fine-tuning on domain-specific data:** Custom terrain datasets (e.g., off-road, Martian terrain) to improve accuracy.
2. **Edge deployment:** NVIDIA Jetson or mobile devices using TensorRT or ONNX optimization.
3. **Video processing:** Frame-by-frame analysis with temporal smoothing for consistency.
4. **Active learning:** Use W&B-logged user images to identify failure cases and retrain.
5. **Absolute depth:** Integrate metric depth models for real-world distance estimation.
6. **Multi-sensor fusion:** Combine with LiDAR or IMU data for more robust navigation.

Q64. Are there ethical concerns?

A.

1. **Misuse in safety-critical systems:** The model is NOT reliable enough for autonomous driving or medical imaging. Over-reliance could cause accidents.

2. **Bias in ADE20k:** The dataset is biased toward certain geographies and cultures (Western urban/suburban scenes). It may underperform in underrepresented regions.
 3. **Privacy:** If used to analyze images of public spaces, it could detect and classify people (the "person" class is a hazard), raising surveillance concerns.
-

16. Rapid Fire / Short Answers

Q65. What programming language did you use?

A. Python 3.10+.

Q66. What deep learning framework?

A. PyTorch (with HuggingFace Transformers library).

Q67. What is the role of `transformers` library?

A. Provides pre-trained SegFormer models and feature extractors via `from_pretrained()`, handles tokenization/preprocessing, and manages model architecture definitions.

Q68. What does `model.eval()` do?

A. Switches the model to evaluation mode — disables dropout layers and uses running batch normalization statistics instead of batch-specific statistics. Critical for deterministic inference.

Q69. What is `torch.nn.functional.interpolate` ?

A. A function to resize tensors using various modes (bilinear, nearest, bicubic). We use bilinear mode to upsample logits from 1/4 resolution to original size.

Q70. What is a feature extractor?

A. `SegformerFeatureExtractor` is a preprocessor that normalizes pixel values (scaling to [-1, 1] or [0, 1] based on ImageNet stats), resizes the image, and converts it to a PyTorch tensor. It ensures the input matches what the model was trained on.

Q71. What is `argmax(dim=1)` ?

A. Takes the index of the maximum value along dimension 1 (the class dimension). For logits of shape `(1, 150, H, W)`, `argmax` along `dim=1` produces a `(1, H, W)` tensor where each value is the predicted class ID (0–149).

Q72. What is morphological closing?

A. A morphological operation: **dilation followed by erosion** with the same kernel. It fills small holes and gaps in binary/labeled regions while preserving overall shape. We use it to connect scattered safe pixels.

Q73. What is Gaussian filtering used for?

A. `scipy.ndimage.gaussian_filter` with `sigma=2` smooths the depth map before computing gradients. This removes high-frequency noise (individual pebble bumps) that would produce falsely high slope values.

Q74. What is the `id2label` dictionary?

A. A mapping from integer class IDs to human-readable class names, stored in the model's config. Example: `{"0": "wall", "1": "building", "9": "grass", "21": "water", ...}`. It has 150 entries for ADE20k.

Q75. What colormap do you use for depth?

A. OpenCV's `COLORMAP_TURBO` — a perceptually uniform colormap where Blue=Low (far), Red=High (near).

Q76. What is the role of `cv2.addWeighted()` ?

A. Blends two images: $\text{dst} = \alpha \times \text{src1} + \beta \times \text{src2} + \gamma$. We use it to blend the original image with the depth heatmap at a user-controlled opacity.

Q77. What are W&B Artifacts?

A. Versioned, tracked files (datasets, models, config) stored in W&B's cloud. In our training script, each checkpoint is logged as an artifact, creating a versioned lineage of model iterations.

Q78. What is the `.env` file for?

A. Stores sensitive credentials (like `WANDB_API_KEY`) outside of code. Loaded at runtime via `python-dotenv`'s `load_dotenv()`. The `.env` file is gitignored to prevent accidental credential leaks.

Q79. What libraries are in `requirements.txt` ?

A. `torch`, `transformers`, `wandb`, `gradio`, `opencv-python`, `pillow`, `numpy`, `tqdm`, `scikit-learn`, `python-dotenv`, `scikit-image`, `scipy`, `plotly`.

Q80. How do you handle errors in model loading?

A. Try-except blocks catch and print errors. If `load_model()` fails, it returns `(None, None, None)`, and `process_image()` checks for `None` and returns an error message to the UI. The app doesn't crash.

Q81. What is `skimage.graph.route_through_array` ?

A. A function from `scikit-image` that finds the **minimum-cost path** through a 2D cost array using Dijkstra's algorithm. It returns the path as a list of (row, col) indices and the total cost. `fully_connected=True` allows 8-directional movement (including diagonals). `geometric=True` weights diagonal steps by $\sqrt{2}$.

Q82. Why is `align_corners=False` used in interpolation?

A. `align_corners=False` aligns pixel centers rather than corners during bilinear interpolation. This is the recommended setting for semantic segmentation as it produces more accurate boundary alignment, especially when upsampling from significantly smaller feature maps.

Q83. What is the significance of the project for autonomous systems?

A. This project demonstrates a complete **perception pipeline** for autonomous navigation:

1. **Scene understanding** (semantic segmentation)
2. **Depth perception** (monocular depth)
3. **Terrain assessment** (safety mapping)
4. **Path planning** (A* pathfinding)
5. **Visualization** (HUD and 3D views)

While not production-ready, it serves as a prototype showing how these components integrate. The modular design allows each component to be upgraded independently (e.g., swapping SegFormer for a domain-specific model).

Q84. Can the safety definitions be changed at runtime?

A. Yes. The Gradio UI includes a JSON code editor where users can modify the safe and hazard keyword lists on the fly. For example, to build a ski navigation system, you could move "snow" from hazard to safe. The mapping is re-parsed on every inference call.

Q85. What is the `debug_checklist.py` file?

A. A diagnostic script that systematically verifies the correctness of the pipeline:

1. Identifies the "person" class ID in ADE20k and confirms it's mapped to hazard.
2. Counts total safe/hazard mapped IDs.
3. Creates a synthetic test with a half-grass, half-person mask and verifies the HUD overlay produces green-left and red-right.
4. Confirms the upsampling logic (bilinear on logits $\rightarrow$ argmax) is correct.

It acts as a **smoke test** to catch regressions.